Context-Aware Classification of Egyptian Hieroglyphs

Maryem Mahmood Zehra Talat

Abstract

Egyptian hieroglyph recognition remains a challenging computer vision and language modelling problem due to the visual ambiguity of signs, the absence of explicit word boundaries, and the contextual nature of hieroglyphic writing. Existing approaches have primarily focused on isolated sign classification using convolutional neural networks (CNNs), while limited work has explored contextual disambiguation. In this study, we investigate the role of sequential context in improving hieroglyph recognition through both statistical and neural methods. We introduce Glyph2025, a merged dataset consisting of 13,287 images across 352 Gardiner sign classes compiled from two public hieroglyph corpora. We first evaluate CNN architectures for isolated glyph classification, where Glyphnet achieves the strongest standalone performance with 82.66% top-1 accuracy. We then incorporate sequential context through an n-gram re-ranking framework combining CNN predictions with statistical language probabilities, achieving up to 88.55% accuracy. Finally, we propose a CNN + Transformer Encoder architecture for contextual sign disambiguation using self-attention. The transformer model achieves 73.51% per-position accuracy, demonstrating that learned long-range contextual reasoning improves recognition performance. Our findings establish sequential context as an important component of robust hieroglyph recognition systems.

1. Introduction

The Egyptian hieroglyphic system is an ancient writing system of the Ancient Egyptians dating to 3000 BC. Jean-Francois Champollion established a historical milestone by deciphering that the hieroglyphic writing system is not a symbolic system—as was believed hitherto—but a complex writing system that combines three distinct categories of signs: logograms, phonograms and semantic classifiers (also known as determinatives). Each of these signs play a key role in terms of their semantic and phonological value: logograms have both a phonological realisation and a semantic value, the phonograms have only a phonological realisation and the classifiers only a semantic value i.e. symbols that represent the object or symbols that narrate the sound (**??**).

The Egyptian hieroglyphic system was not a straightforward writing system, with one sign having several different phonetic values, and several signs having the same phonetic value; signs were combined, and interpretants—phonetic complements—were also used. Through the text, we could not understand word boundaries nor the end of sentences. Hieroglyphic scripture varied from scribe to scribe and there was no indication of word boundaries or end of sentences. Thus, producing high quality machine readable hieroglyphic texts remains an open problem within the field (**?**). The Gardiner's Sign List was developed in 1927 by Sir Alan Gardiner and has become an international standard system for classifying Ancient Egyptian Hieroglyphs. It provides an organisation of hieroglyphs within categories based on their meaning: each sign is given a letter for the category it falls under and a number for the individual sign (**?**). It is the main reference system we use in our research.

Barucci et al. developed a custom designed CNN, "Glyphnet", for the visual recognition and classification of Egyptian hieroglyphs. The Glyphnet architecture surpassed existing CNN architectures (ResNet-50, Inception-v3, and Xception) which had been tested using both transfer learning and training from scratch. They trained and evaluated their model on a merged dataset which consists of Franken and van Gemert's Pyramids of Unas dataset—with 4,310 grayscale images across 172 classes, taken from images of the walls inside the pyramid—and a custom dataset of 1,310 coloured images of 48 different hieroglyph sourced from different materials and historical periods, in order to improve the model's generalisation capabilities. They selectively merged the two datasets to include only the hieroglyph classes in common, resulting in a dataset of 4,309 images distributed across 40 distinct classes.

They used the merged dataset to train the ResNet-50 model—which used "residual mapping" to solve the exploding and vanish gradient problem—the Inception-v3—a many layered network using parallel convolution layers with different kernel sizes—and the Xception—which replaces standard inception modules with depthwise separable con-

volutions. The lightweight Glyphnet, with only 494,504 trainable parameters (a massive downsizing from the millions of parameters in the classical networks) and relying on separable convolutional layers, was created by the authors specifically for the task of hieroglyphic recognition in order to reduce computational complexity and avoid the overfitting issues for the relatively small dataset. The Glyphnet model achieved an accuracy of 0.976, precision of 0.975, recall of 0.965, and an F1-Score of 0.968; it was the fastest network in terms of training and prediction times. Thus, this paper provided foundational research in single hieroglyph recognition which we further altered for the purposes of our study.

Sequential context is another area of current hieroglyphic research. Franken and von Gemert introduce context within the hieroglyphic framework by testing a lexicon lookup (matching sequences to a dictionary of existing words) and an N-gram model (calculating the probability of a specific sequence of symbols occurring in a row). Unlike other models, the Franken model is a statistical language model, which reranks the visual matches based on context. The model was evaluated on two kinds of datasets—a visual set of photographs for image recognition and a textual corpus for the language model training. The visual data consisted of 10 photographic plates taken from the pyramid of Unas—a single pyramid was used to avoid complications arising from different dialectic writing styles. Across the visual set, there are a total of 3,993 individual hieroglyphs of 161 unique hieroglyph types which were manually segmented and labelled. Labelling was done with reference to an open-source JSesh project. The textual corpus also originated from the JSesh project, which consisted of 158 pyramid texts.

Franken and von Gemert's model follows a pipeline with visual and textual components to recognise hieroglyphs with the added factor of context. A saliency-based text-detection algorithm was used which outputs an unordered list of bounding boxes, which were sorted to establish the correct sequence for hieroglyphic reading. The system isolated symbols by applying approximate texture to the overlapping sections. After isolating symbols, the authors used five different descriptors to evaluate the extracted visual features by the model: Shape-Context (SC) and Self-Similarity (SS) based on shape, Histogram of Oriented Gradients (HOG) based on appearance, and a combination of descriptors which mixed shape and appearance features, HOOSC (HOG + SC) and HOOSS (HOG + SS). These visual descriptors were compared against the labelled training set. The language model then re-ranked the previous outputs by introducing context, in order to fix visual errors. It used statistical language models pre-trained on separate training data to extract the top visual matches for each isolated symbol and re-rank them accordingly. This is done through a two fold process: Lexicon Lookup which attempts to match consecutive hieroglyphs to known words in a dictionary, weighting them by word length and occurrence probability and N-gram Model, which uses statistics to determine the probability of a specific sub-sequence of hieroglyphs appearing in a row. The text detection algorithm was able to locate 83% of manually annotated hieroglyphs of which 85.5% were correct, based on a predecided criteria. Adding context via the N-gram language model improved the average recognition results by 1%, which is a benchmark our paper seeks to reproduce and improve.

To further the contextual understanding of hieroglyphs for improved classification, we incorporate current advanced research within the domain, notably in the form of the transformer (**?**). The research gap we identify is the disconnect between visual identification and linguistic understanding. Prior work has successfully named signs but struggles to interpret them in sequence. There is no existing framework that uses Attention to resolve visual ambiguities (like "Bird-type" errors) through long range linguistic context. Additionally, current models are unable to distinguish between signs and "scribal hand" variations. We therefore propose to investigate a Vision Language Transformer approach. Prior work does classification using CNN; we therefore propose to investigate Contextual Sign Disambiguation via Attention. By treating a row of hieroglyphs as a sequence rather than a bag of objects, we can leverage neighbouring symbols to infer the identity and meaning of ambiguous signs. We therefore delve into the use of Multi-Head Attention mechanism to improve the recognition of ambiguous hieroglyphs by leveraging long-range linguistic context, and to effectively normalize variations across different scribal hands.

Our study first provides a comprehensive evaluation of CNN architectures on Glyph2025, establishing that pretrained backbones with fine-tuning offer a robust baseline, with Glyphnet (82.66%) significantly outperforming ResNet-18 (67.08%). We further demonstrate that statistical sequential context aids visual classification through an n-gram re-ranking pipeline combining CNN predictions with trigram probabilities. Additionally, our transformer encoder architecture achieves 73.51% per-position accuracy, marking a 6.43% improvement over isolated CNNs. Finally, a progressive fine-tuning strategy—moving from the classifier head to the transformer encoder and then the CNN backbone—reaches a peak accuracy of 86.45%, representing a +12.94% gain over jointly-trained transformers and a +19.37% improvement over the CNN baseline.

Property	Value
Total images	13,287
Total classes (raw)	352 (310 EHT + 42 Franken-unique)
File formats	10,696 PNG + 2,591 JPG
Most common class	N35 (water ripple) — 942 images
Least common class	L7 — 1 image
Median images per class	6
Mean images per class	38
Classes with < 5 images	153 (long-tail problem)

Table 1. Glyph2025 dataset statistics

2. Methodology

2.1. Dataset

We trained our model on a custom merged dataset, Glyph2025. Our dataset merged two publicly available hieroglyph image sets, with applied data augmentation into a single dataset, providing a streamlined version to act as a unified benchmark. The first image set source was Ferrer's Egyptian Hieroglyphic Text (EHT), containing 9,703 images across 310 classes, downloaded from Github (**??**). The EHT dataset consists of diverse sources—carved stone, painted tomb walls, papyrus manuscripts and line drawings; this allows for better generalisation across different styles but also causes extreme domain shift between symbols of the same class. The second image set is sourced from Franken and van Gemert's work, downloaded from Github (**??**). This consists of 4,210 manually annotated images across 171 classes. We exclude the UNKNOWN labeled images (179 in number). Each image filename encodes plate number and reading-order index. This reading-order metadata is crucial because it provides the sequential structure utilised in Phases 2 and 3 that the EHT images lack.

Since both datasets used Gardiner sign codes as folder names, the merge is relatively straightforward: combining all overlapping classes—42 classes in Franken's dataset were not present in EHT but were added to Glyph2025, adding depth to our combined dataset. As a result, Glyph2025 consists of 13,287 total images with 352 classes.

Despite a greater dataset size, Glyph2025 has its own limitations. There is severe imbalance, inherited from the two source datasets. A small subset of high-frequency phonograms including (N35, M17, G1, G43, D21) dominate the dataset, while the majority of classes are starved of data; with a median of 6 images per class and a mean of 38, with some classes having as few as 1 image. This can lead to severe problems where a model would spend overwhelmingly more time learning common signs and effectively ignore the long tail. We address this using WeightedRandomSampler on the training DataLoader (sampling weight = $1/\text{class\_frequency}$) so rare classes appear as often as common ones in each mini-batch, and weighted CrossEntropyLoss:

$$\text{weight}_c = \frac{N_{\text{total}}}{N_{\text{classes}} \times N_c}$$

giving rare classes stronger gradient signals.

Inherent within the dataset are also problems of domain shift and varying scribes. Although the Gardiner code is the same, the symbols may have different physical supports which are completely dissimilar at the pixel level. Using a model pretrained on ImageNet features is beneficial as it already understands edges, textures, and shapes without reliance on the medium. Within the same physical support code, scribal differences arise: different proportions, stroke thickness, and level of detail. This in-class variation makes it difficult for the model to distinguish and classify through visual features alone. Here, we incorporate sequential context to aid the model.

We filtered the dataset on running to extract classes with ≥ 7 samples, after which we got 12,836 images across 176 classes. The filtering threshold ensured we had enough samples per class to divide into train, test and validation sets. However, the filtering resulted in dropping 176 classes with 451 images collectively—however we theorise this does not cause an error as the rare classes would not be well learned and statistically unreliable to evaluate. The stratified 70/15/15 split produces: 8,985 train / 1,925 val / 1,926 test. Stratification ensures every class appears proportionally in all three splits.

We incorporated feature enrichment within Glyph2025 through dataset annotation derived from the Gardiner Sign List, which includes metadata fields for functional role (phonogram, logogram, and determinative mapped to the category prefix), phoneme count (labelled as uniliteral / biliteral / unknown based on consonant representation), and class rarity (labelled as common / mid / rare based on corpus frequency from the n-gram data). During training, data augmentation and transformation was applied on the fly. Validation and test set transformations were deterministic and were not annotated, ensuring reproducible results.

2.2. Baseline Models

2.2.1. Phase 1: CNN

Initially we explored the ConvNeXt-Tiny with 28.6M parameters, pretrained on ImageNet. It was fine-tuned on Glyph2025 for 60 epochs with CosineAnnealingWarmRestarts. This achieved an accuracy of only 56.23% top-1

Table 2. Data augmentation and preprocessing pipeline

Transform	Parameter	Justification
Resize	256×256	Upsample from variable input sizes (mean 117×104 px)
RandomResizedCrop	224×224, scale=(0.85, 1.0)	Simulates zoom variation across photograph sources
RandomHorizontalFlip	$p = 0.5$	Hieroglyphs can face either direction without changing meaning
RandomRotation	$\pm 15°$	Compensates for inscriptions carved at slight angles
ColorJitter	brightness = 0.3, contrast = 0.3	Handles lighting variation between sources
ToTensor + Normalize	ImageNet mean/std	Required for pretrained ResNet-18 backbone

and 52.72% macro F1. The overfitting during training was caused by the model's large size (28.6M parameters) compared to the smaller dataset (12,836 images).

For improved baseline results, we explored the Glyphnet model designed by Barucci et al specifically for hieroglyph recognition. Glyphnet was a comparatively lightweight CNN model with separable convolutions totalling ∼499K parameters. It outperformed the ConvNeXt-Tiny significantly on Glyph2025, with an accuracy of 82.66% top-1 and 67.68% macro F1.

Despite Glyphnet's success as the baseline model with a high raw accuracy, it wasn't able to translate into improved accuracy in the successive baseline models. Thus we adopted ResNet-18 as the final backbone, with 11.2M parameters and pretrained it on ImageNet.

This decision was motivated by three main factors:

1. ResNet's 512 dimensional feature space, trained on ImageNet, provides richer and transferable representations, resulting in a higher feature quality than Glyphnet's model trained from scratch—which is required by downstream tasks particularly transformer context.

2. It provides a degree of standardisation as ResNet-18 is an established baseline model and makes our results comparable to other works and reproducible.

3. ResNet-18's layered architecture allows for progressive freezing and fine-tuning unlike Glyphnet's block structure.

Although ResNet-18's top-1 accuracy of 67.08% is lower than Glyphnet, it has a suitable top-5 accuracy (93.61%) which is what is required by the following phases.

2.2.2. Phase 2: CNN and N-gram Re-ranking

We discussed previously that hieroglyphic signs can be indicative of several types or functions—logograms, phonograms or determinatives. The function of a particular symbol depends on the surrounding symbols (context). Thus, a CNN, which isolates symbols and classifies them independent of context, does not give an accurate depiction of the symbol.

Table 3. CNN backbone comparison

Metric	ConvNeXt-Tiny	Glyphnet	ResNet-18
Top-1 Accuracy	56.23%	82.66%	67.08%
Top-5 Accuracy	—	96.16%	93.61%
Macro F1	52.72%	67.68%	62.83%
Parameters	28.6M	499K	11.2M
Input	224×224 RGB	100×100 grey	224×224 RGB
Pretrained	ImageNet	No (scratch)	ImageNet

We address this by combining the output of CNN (visual representations) with statistical language patterns learned from a different corpus of ancient Egyptian texts. The CNN used is the ResNet-18 backbone from Phase 1, frozen—no training occurs, and the language model is built from precomputed corpus statistics, not learned. We tune a single parameter alpha to control the balance between visual output (from frozen ResNet-18) and context.

The linguistic information is present in `nGrams.txt` and contains pre-computed unigram, bigram, and trigram frequencies extracted from 158 Pyramid Texts encoded in the JSesh database. The textual information consists purely of Gardiner code sequences with no images. Thus these two streams of data are able to capture different kinds of knowledge. Combining both sources resolves ambiguities that neither could handle alone.

The sequences encoded within the file contain unigram counts of each symbol independently, bigram counts indicating how often each sign follows another, and trigram counts representing conditional continuation probabilities. We compute conditional probabilities through Laplace Smoothing:

$$P(C|B) = \frac{\text{count}(B, C) + \lambda_{bi}}{\text{count}(B, *) + \lambda_{bi}|V|}$$

$$P(C|A, B) = \frac{\text{count}(A, B, C) + \lambda_{tri}}{\text{count}(A, B, *) + \lambda_{tri}|V|}$$

where $|V|$ is the vocabulary size, $\lambda_{bi} = 1/200$, and $\lambda_{tri} =$

$1/2000$.

Laplace Smoothing allows us to account for unseen yet valid sign combinations, without giving them a probability of 0—which would block the n-gram from selecting it. We apply a back-off strategy as well where if the trigram context (A, B) was unseen, then it reverts to the bigram context of $P(C|B)$.

The re-ranking pipeline constitutes two steps:

1. **Visual scoring**: the frozen ResNet-18 processes the glyph image and produces a softmax probability distribution over 176 classes.

2. **N-gram reranking**: a combined score is calculated:

$$S_j = v_j^{\alpha} \times P(c_j|\text{context})$$

where v_j is the CNN softmax score for candidate j, α is the visual exponent, and $P(c_j|\text{context})$ is either the trigram or bigram probability.

Since Glyph2025 contains isolated images, evaluation sequences were constructed via weighted random walks through the trigram graph. This produces 2,000 sequences averaging 18.6 signs each (37,143 total evaluation positions), split 50/50 into validation and test sets.

Table 4. N-gram re-ranking results

α	Visual	+Bigram	Δ	+Trigram	Δ
1.0	71.92%	88.09%	+16.17%	88.55%	+16.62%
2.0	71.92%	84.79%	+12.86%	87.68%	+15.76%
3.0	71.92%	82.20%	+10.28%	85.27%	+13.35%
4.0	71.92%	80.24%	+8.32%	83.06%	+11.14%
5.0	71.92%	79.10%	+7.18%	81.28%	+9.36%
6.0	71.92%	78.16%	+6.24%	79.83%	+7.91%

The best $\alpha = 1.0$ means the n-gram model has substantial influence over the final prediction. Since in the earlier phase we see the CNN is at a relatively lower accuracy of 67%, the visual model is benefitted by the language model.

2.3. 2.2.3 Phase 3: CNN + Transformer Encoder

While the n-gram re-ranking approach in Phase 2 demonstrated that sequential context improves hieroglyph recognition, it suffers from two major limitations. First, trigram models rely on a fixed context window of only two previous signs, whereas Egyptian hieroglyphic writing often contains long-range dependencies. Determinatives, for example, may appear several positions away from the phonograms they modify. Second, n-gram probabilities are static corpus statistics and cannot learn adaptive contextual reasoning tailored to the visual ambiguities produced by the CNN backbone.

To address these limitations, we implemented a Transformer Encoder architecture capable of modelling long-range dependencies through self-attention. Unlike n-gram models, the transformer allows every sign in a sequence to attend to every other sign simultaneously, enabling contextual disambiguation across the full inscription.

The architecture consists of four main components. First, each hieroglyph image is passed independently through the frozen ResNet-18 backbone from Phase 1. Instead of using the final classification logits, we extract the 512-dimensional feature vector prior to the classifier layer. Since the backbone remains frozen throughout training, features are pre-extracted and cached, significantly reducing training time.

Second, the extracted features are projected into a transformer embedding space of dimension $d_{model} = 256$ using a learned linear projection followed by Layer Normalization and dropout. Learned positional embeddings are then added so that the model can distinguish the order of glyphs within the sequence.

Third, the contextual modelling is performed using a 3-layer Transformer Encoder with 4-head self-attention. Each encoder block contains multi-head attention, residual connections, Layer Normalization, and a feedforward network with GELU activation. Through self-attention, each position incorporates information from the entire sequence, allowing the model to learn contextual relationships between distant hieroglyphs. Padding masks are applied so that padded positions do not influence the attention computation.

Finally, a linear classification head maps the contextualized representations to probability distributions over the 176 hieroglyph classes. Training uses cross-entropy loss with label smoothing and ignores padded positions during optimization.

Unlike autoregressive transformer decoders, we employ a bidirectional encoder architecture because the task is sequence labelling rather than text generation. Both preceding and following signs provide useful contextual information for hieroglyph classification, making full bidirectional attention more appropriate.

Training sequences were constructed from the Franken subset of Glyph2025, where filenames encode plate number and reading order. Images were grouped by plate, sorted according to glyph index, filtered to valid classes, and chunked into sequences of up to 50 signs. To improve visual diversity, images corresponding to each Gardiner code were randomly sampled from the entire Glyph2025 dataset during training rather than using a fixed image per position.

The transformer was trained using AdamW optimization with a learning rate of 5×10^{-4}, cosine decay scheduling, and 5-epoch warmup. Training was performed for up to 40

epochs with early stopping based on validation accuracy.

The transformer encoder achieved a per-position accuracy of 73.51%, representing a 6.79% improvement over isolated CNN classification. This confirms that learned sequential context improves hieroglyph recognition. Although macro F1 decreased due to poorer performance on rare classes, weighted F1 improved, indicating stronger performance on the most frequent and visually ambiguous signs. Compared to the n-gram model, the transformer provides a more flexible and generalizable framework for contextual sign disambiguation.

2.4. Deliverable 4: Progressive Fine-tuning

Having established the CNN + Transformer pipeline in Phase 3 (73.51% per-position accuracy), we turn to improving the model without changing its architecture. The central question is whether the Phase 3 result represents the limit of what a 3-layer transformer can learn from 52 training sequences, or whether a better training strategy can extract more from the same model and data.

Our first approach borrowed directly from Phase 1, where freezing the ResNet-18 backbone and training only the classifier head before unfreezing Layer 4 had improved accuracy from $\sim$22% to 67%. We hypothesised that the same principle would transfer to the transformer: train the classifier head first to establish stable gradients, then unfreeze the encoder layers.

Stage A froze the input projection, positional embeddings, and all three encoder layers, leaving only the Linear$(256 \rightarrow 176)$ classifier trainable (45,232 parameters). We trained for 10 epochs at LR $= 10^{-3}$.

Stage B unfroze all 1,770,672 parameters and continued training at LR $= 5 \times 10^{-5}$ with warmup and cosine decay, using early stopping with patience 8.

The results were disappointing. Stage A hit 100% training accuracy by epoch 3 while validation stayed at 73.51%—exactly the Phase 3 baseline. Stage B oscillated around 73–74% validation accuracy for 20 epochs, eventually settling at 73.71%: a +0.20% improvement that falls within noise. Worse, macro F1 dropped from 49.90% to 48.41%, meaning the fine-tuning had slightly degraded performance on rare classes while providing negligible gains elsewhere.

The training curves told a clear story. With 100% training accuracy from the start of Stage B, the model had already memorised the 52 training sequences during Phase 3. The uniform learning rate of 5×10^{-5}, while small, was still aggressive enough to push the already-converged encoder layers away from their Phase 3 optimum without finding a better one.

2.4.1. Diagnosing the failure

The progressive unfreezing strategy rests on an assumption that did not hold here. In Phase 1, the frozen layers contained *ImageNet weights*—pretrained representations from a different domain that had never seen a hieroglyph. The classifier head genuinely needed to learn a new mapping, and the backbone genuinely needed task-specific adaptation. Freezing and unfreezing made sense because the layers started in different states of readiness.

The Phase 3 transformer was different. All layers had been trained *jointly* for 40 epochs on the same hieroglyph sequence labeling task. The classifier, encoder layers, projection, and positional embeddings had co-adapted—each layer's weights implicitly depended on the others. Freezing the encoder and retraining only the classifier disrupted this co-adaptation: the classifier shifted to accommodate the frozen (random-looking, from its perspective) encoder outputs, and when the encoder was subsequently unfrozen, the two components were no longer aligned.

This diagnosis pointed to a different strategy: rather than freezing and unfreezing, allow all layers to update simultaneously but at *different rates*, preserving the co-adapted representations while still permitting refinement.

The enhanced Deliverable 4 replaced the uniform learning rate with Layer-wise Learning Rate Decay (LLRD), a technique established in the BERT fine-tuning literature (**??**). The principle is that layers closer to the input encode more general, reusable features and should change conservatively, while layers closer to the output are more task-specific and can adapt more freely.

We assigned each layer group a learning rate that decays by a factor of $\sim$0.75 per layer from the classifier head downward:

Stage A used the same classifier-only setup as the original attempt but with stronger regularisation (increased label smoothing) to prevent the immediate 100%-training-accuracy collapse. Stage B then applied LLRD across all layers for 20 epochs with warmup and cosine decay.

The difference in training dynamics was immediate. Where the original Stage B showed 100% training accuracy from epoch 1 with validation oscillating, the enhanced Stage B maintained a gap between training and validation accuracy that gradually narrowed—the hallmark of a model that is learning rather than memorising. The deeper layers changed slowly enough to preserve the Phase 3 representations while the classifier and upper encoder layers adapted to produce more refined predictions.

2.4.2. Evaluation methodology

A key shortcoming of the original D4 was that it compared its test accuracy (73.71%) against Phase 3's reported num-

Table 5. Phase 3 Results: CNN vs CNN + Transformer Encoder

Metric	CNN Only	CNN + Transformer	Improvement
Per-Position Accuracy	66.72%	73.51%	+6.79%
Macro F1	62.05%	49.90%	-12.15%
Weighted F1	68.30%	71.07%	+2.77%
Sequence Accuracy	N/A	0.00%	–

Table 6. LLRD configuration for D4 Enhanced.

Layer Group	Parameters	Learning Rate
Input proj + positional	144,128	1.58×10^{-5}
Encoder layer 0	527,104	2.11×10^{-5}
Encoder layer 1	527,104	2.81×10^{-5}
Encoder layer 2	527,104	3.75×10^{-5}
Classifier head	45,232	5.00×10^{-5}

ber (73.51%) without controlling for evaluation conditions. Differences in random image sampling during sequence construction, data loader ordering, or even GPU numerical precision could account for a 0.20% gap.

The enhanced D4 addresses this with a rigorous three-way evaluation. We loaded the original D4 checkpoint, the enhanced D4 checkpoint, and the Phase 1 ResNet-18, then evaluated all three on the *exact same test sequences with the exact same images*. This eliminates all confounds except model quality. We further decomposed results by class frequency (rare vs. common), computed per-sequence accuracy and head-to-head win counts, calculated corpus-level BLEU scores to measure multi-glyph coherence, and examined specific classes where models diverged.

This diagnostic depth—absent from the original D4—transforms the deliverable from a simple "did accuracy go up?" exercise into a genuine analysis of what the transformer learns, where it succeeds, and where the data limits further progress.

3. Results

We evaluate our pipeline across five experimental phases on the Glyph2025 dataset (176 classes, 12,836 images). All phases use the same stratified 70/15/15 split (random seed 42) and report metrics on the held-out test set unless otherwise noted. Phase 2 uses n-gram–derived evaluation sequences (1,000 test sequences, 18,555 positions); Phases 3 and 4 use Franken plate reading-order sequences (15 test sequences, ∼600 positions). We note this distinction explicitly where comparisons are drawn.

3.1. Phase 1: CNN Baseline

Table 7 summarises our architecture exploration. We evaluated three backbones on Glyph2025 to select the strongest foundation for downstream contextual models.

ConvNeXt-Tiny's 28.6M parameters severely overfitted the 12,836-image dataset, yielding only 56.23%. Glyphnet, purpose-built for hieroglyphs with just 499K parameters, achieved the highest raw accuracy (82.66%) but operates on a grayscale 100×100 pipeline incompatible with standard pretrained backbones. We selected ResNet-18 as the final backbone despite its lower top-1 because (a) its 512-dimensional ImageNet-pretrained feature space provides richer representations for the transformer in Phase 3, (b) its layered architecture enables the progressive unfreezing strategy central to Deliverable 4, and (c) the 93.61% top-5 accuracy ensures the correct class is almost always among the CNN's top predictions—the ceiling that Phases 2 and 3 can exploit.

The two-phase training strategy proved essential: head-only training (Phase 3a, 10 epochs) reached 28.16% validation accuracy, establishing a stable classifier initialisation before Layer 4 fine-tuning (Phase 3b) pushed the model to 67.08%. Per-category analysis (Table 8) reveals that visually distinctive categories (X: 79.7%, O: 79.5%) perform well, while categories containing many similar-looking signs—particularly M (reeds, 31.4%) and G (birds, 51.7%)—remain challenging without sequential context.

3.2. Phase 2: N-gram Re-ranking

Table 9 presents the alpha tuning results on 1,000 validation sequences (18,588 positions). The optimal $\alpha = 1.0$ indicates substantial reliance on the language model, consistent with the CNN's moderate base accuracy.

On held-out test sequences (Table 10), trigram re-ranking achieved 89.21% per-position accuracy, a +16.40% improvement over visual-only. The trigram outperformed the bigram (+0.78%), confirming that the additional sign of context helps. However, we note that these evaluation sequences were constructed via random walks through the trigram graph from `nGrams.txt`—the same data source used to build the n-gram model—which partially inflates the absolute improvement.

3.3. Phase 3: CNN + Transformer Encoder

Table 11 compares Phase 3 against the Phase 1 baseline on Franken plate sequences (15 test sequences, real inscription reading order). The transformer achieves a +6.79% per-

Table 7. Phase 1 CNN architecture comparison on Glyph2025 (176 classes).

Model	Top-1 (%)	Top-5 (%)	Macro F1 (%)	Params
ConvNeXt-Tiny	56.23	–	52.72	28.6M
Glyphnet (**?**)	82.66	96.16	67.68	499K
ResNet-18 (final)	67.08	93.61	62.83	11.2M

Table 8. Phase 1 per-Gardiner-category test accuracy (top categories by sample count).

Category	Test Samples	Accuracy (%)
X (bread/offerings)	123	79.7
O (buildings)	83	79.5
S (crowns/clothing)	115	75.7
D (body parts)	307	68.4
N (sky/water/land)	233	67.0
V (rope/fibre)	157	66.9
G (birds)	147	51.7
M (plants/reeds)	185	31.4

Table 9. Phase 2 alpha tuning on validation sequences. Visual-only accuracy is constant across α.

α	Visual (%)	+ Bigram (%)	+ Trigram (%)	Δ_{tri}
1.0	71.92	88.09	88.55	+16.62
2.0	71.92	84.79	87.68	+15.76
3.0	71.92	82.20	85.27	+13.35
4.0	71.92	80.24	83.06	+11.14
5.0	71.92	79.10	81.28	+9.36
6.0	71.92	78.16	79.83	+7.91

position accuracy improvement, confirming that learned full-sequence context aids recognition on independent evaluation data.

The macro F1 drop (-12.15%) indicates that the transformer improved common classes at the expense of rare ones, which appear in too few of the 52 training sequences to learn reliable contextual patterns. The weighted F1 (+2.77%), which accounts for class frequency, confirms net improvement. The 0.00% sequence accuracy reflects the difficulty of predicting all positions correctly in sequences averaging 42 signs ($0.7351^{42} \approx 0.003\%$).

3.4. Deliverable 4: Progressive Fine-tuning

3.4.1. Phase 4: Original attempt

The first fine-tuning attempt applied progressive unfreezing (classifier head first, then full transformer) to the Phase 3 checkpoint. Stage A achieved 100% train accuracy by epoch 3 while validation remained at 73.51%, and Stage B produced only a +0.20% improvement (Table 13). The macro F1 decreased by 1.49 percentage points, indicating slight degradation on rare classes.

3.4.2. Phase 5: Enhanced attempt (LLRD)

The enhanced version replaced the uniform learning rate with Layer-wise Learning Rate Decay (LLRD), assigning progressively lower rates to deeper transformer layers (Table 12). This prevents deeper layers—which encode more general representations—from being overwritten as aggressively as the task-specific classifier head.

Table 13 presents the three-way comparison on identical test sequences, ensuring differences reflect model quality rather than data loading variations.

3.4.3. Common vs. rare class analysis

Table 14 stratifies the F1 improvement by class frequency. The enhanced D4 produces a +3.53% F1 improvement on common classes (those above the 20th percentile of training frequency) and rescues several specific signs—W18, G4, D2, P1, and G36—where the original D4 had F1 $= 0$. Rare classes (≤ 3 training occurrences) show no improvement across any model, confirming this as a data limitation rather than a model failure.

3.4.4. Sequence-level evaluation

Table 15 reports corpus-level BLEU scores, treating predicted Gardiner code sequences as text translations. The improvement grows from BLEU-1 (+1.59) to BLEU-3 (+2.12), indicating that the enhanced model is specifically better at producing correct multi-glyph subsequences—the expected benefit of improved attention patterns.

In the per-sequence head-to-head comparison, the enhanced D4 beats the original D4 on 5 sequences, loses on 3, and ties on 8. Against ResNet-18, the transformer wins on 14 sequences, loses on 0, and ties on 2—a clean sweep confirming that contextual models consistently outperform isolated classification regardless of sequence length or difficulty.

3.5. Full pipeline summary

Table 16 consolidates results across all phases. We emphasise that Phase 2 and Phases 3–4 use different evaluation protocols (n-gram–derived vs. Franken plate sequences) and are therefore not directly comparable in absolute terms.

Table 10. Phase 2 test results ($\alpha = 1.0$, 1,000 sequences, 18,555 positions).

Method	Per-pos Acc	Macro F1	Seq Acc
Visual only (CNN)	72.81	64.05	1.40
+ Bigram	88.43	80.56	—
+ Trigram	89.21	79.33	—

Table 11. Phase 3 test results on Franken plate sequences.

Model	Per-pos Acc	Macro F1	Weighted F1	Seq Acc
Phase 1: ResNet-18	66.72	62.05	68.30	N/A
Phase 3: + Transformer	73.51	49.90	71.07	0.00
Δ	+6.79	-12.15	+2.77	—

Table 12. LLRD configuration for D4 Enhanced Stage B.

Layer Group	Parameters	Learning Rate
Input proj + positional	144,128	1.58×10^{-5}
Encoder layer 0	527,104	2.11×10^{-5}
Encoder layer 1	527,104	2.81×10^{-5}
Encoder layer 2	527,104	3.75×10^{-5}
Classifier head	45,232	5.00×10^{-5}

4. Discussions

4.1. Why progressive unfreezing failed but LLRD succeeded

The original Deliverable 4 applied the same progressive unfreezing strategy that succeeded in Phase 1 (head-only training followed by backbone unfreezing). In Phase 1, this was effective because the frozen layers were *pretrained on a different task* (ImageNet classification)—the head needed to learn the hieroglyph-specific mapping before the backbone could safely adapt. However, the Phase 3 transformer was already *jointly converged*: all layers had been trained together for 40 epochs to minimise the same sequence labeling loss. Freezing the encoder and retraining only the classifier disrupted this joint optimisation without providing sufficient new learning signal to recover.

The enhanced D4's Layer-wise Learning Rate Decay (LLRD) respects the existing convergence by allowing all layers to update simultaneously but at different rates. The input projection and early encoder layers—which encode more general sequence representations—change slowly (1.58×10^{-5}), while the classifier head adapts faster (5.00×10^{-5}). This mirrors the principle from ULMFiT (**?**) and BERT fine-tuning (**?**): deeper layers should be more conservative because their features are reused by all layers above them. The result is a +1.19% accuracy and +1.53% macro F1 improvement—modest in absolute terms but achieved *without any architectural change*, confirming that the training strategy was the bottleneck, not the model capacity.

4.2. Where the improvement concentrates

The +3.53% F1 improvement on common classes (Table 14) with zero improvement on rare classes (≤ 3 training occurrences) reveals a clear pattern. The original D4's aggressive fine-tuning had caused *catastrophic forgetting* on several medium-frequency classes—W18, G4, D2, P1, and G36 all collapsed to F1 $= 0$. The LLRD approach preserved the transformer's ability to classify these signs by limiting how much the deeper layers could change. This is a training stability result, not a representational one: the model architecture is identical between the two D4 variants; only the learning rate schedule differs.

Rare classes remain at noise-level F1 across all three models (ResNet-18, D4 Original, D4 Enhanced). Per-class F1 analysis shows that model performance stabilises only above approximately 10 training occurrences. Below this threshold, the model's behaviour is dominated by the small sample size rather than learned representations. This is a fundamental data limitation of the 52-sequence training corpus: with an average of 42 signs per sequence, many of the 176 classes appear only 1–3 times across all training data.

4.3. The case for sequential context

The most compelling evidence for sequential context comes from the three-way evaluation on identical test sequences (Table 13). ResNet-18, running each image independently with no contextual information, achieves only 48.41% per-position accuracy on the Franken plate sequences—substantially lower than its 66.72% on the stratified test set. This drop occurs because plate sequences contain a different class distribution with more rare signs and longer contextual dependencies than the balanced test set.

The enhanced D4 transformer achieves 74.01% on these same sequences—a +25.60% absolute improvement. This gap demonstrates that learned sequential context does not merely help marginally; it fundamentally changes the model's ability to handle real inscription data. Without context, nearly half the predictions are wrong; with context, three-quarters are correct.

Table 13. Deliverable 4: three-way comparison on identical Franken plate test sequences.

Model	Per-pos Acc	Macro F1	Weighted F1
ResNet-18 (no context)	48.41	34.57	54.28
D4 Original	72.82	46.57	69.77
D4 Enhanced (LLRD)	74.01	48.10	71.81
Δ Enhanced vs Original	+1.19	+1.53	+2.04
Δ Enhanced vs ResNet	+25.60	+13.53	+17.53

Table 14. Macro F1 stratified by class frequency (20th-percentile threshold = 3 occurrences).

Stratum	n	ResNet (%)	D4 Orig (%)	D4 Enh (%)	$\Delta_{\text{Enh-Orig}}$
Rare (≤ 3)	18	29.54	5.56	5.56	+0.00
Common (> 3)	62	58.34	58.47	62.00	+3.53

The BLEU score analysis (Table 15) provides additional evidence. The improvement grows from BLEU-1 (+1.59) to BLEU-3 (+2.12), indicating that the enhanced model is specifically better at producing correct *multi-glyph subsequences*. This is precisely what self-attention should improve: by attending across the sequence, the transformer ensures that adjacent predictions are mutually consistent, not just individually plausible. The qualitative examples in Deliverable 4 confirm this—in a 50-sign sequence, the transformer correctly predicts 37 positions while ResNet-18 manages only 19, with the transformer's errors concentrated on rare signs while ResNet-18 fails even on common signs that context disambiguates easily.

4.4. Reconciling Phase 2 and Phase 3 results

Phase 2 (n-gram re-ranking) shows a larger raw improvement (+16.40%) than Phase 3 (transformer, +6.79%), but these numbers are not directly comparable due to different evaluation protocols. Phase 2 evaluated on sequences constructed from the same `nGrams.txt` file used to build the n-gram model, creating a partially circular evaluation. The optimal $\alpha = 1.0$—indicating the language model dominates the visual model—is symptomatic of this: the n-gram can predict its own sequences with high confidence regardless of visual evidence.

Phase 3 evaluated on real Franken plate sequences with ground-truth reading order, a substantially harder and more honest test. The transformer's +6.79% improvement (and D4 Enhanced's +7.29% vs Phase 1) on these independent sequences represents genuine learned contextual reasoning. On the same test sequences evaluated identically, the transformer outperforms ResNet-18 by +25.60%—the most reliable measure of how much context helps.

A standardised evaluation protocol using a single sequence source across all phases would strengthen future comparisons.

4.5. Limitations

Several limitations constrain the conclusions that can be drawn from this work.

First, the **sequence training corpus is very small**: 52 training sequences from 10 plates of a single pyramid. This limits the transformer's ability to learn complex attention patterns and is the primary cause of the rare-class failure. Expanding the corpus using the rosmord/MDC-texts repository (**?**) (containing hundreds of additional inscriptions) is the most impactful direction for future improvement.

Second, **macro F1 remains well below accuracy** across all contextual models (48.10% macro F1 vs 74.01% accuracy for D4 Enhanced). The class-frequency analysis confirms this is driven by the long tail: 18 classes with ≤ 3 training occurrences contribute disproportionately to the macro average while being statistically unreliable. Reporting median per-class F1 or filtering to classes above a minimum occurrence threshold would provide a more stable metric for future work.

Third, the **Phase 2 evaluation is partially circular**. While it successfully demonstrates the principle that n-gram context aids recognition, the absolute improvement numbers (89.21% accuracy) overstate the benefit. An independent evaluation using Franken plate sequences for the n-gram pipeline would provide a fairer comparison with the transformer results.

Fourth, **the Franken reading order may contain annotation noise**. The location files were manually created by a non-Egyptologist, and some reading-order assignments may be incorrect, introducing label noise into the sequence data. Verification by an Egyptologist would improve data quality.

5. Conclusion

This study investigated the role of sequential context in Egyptian hieroglyph recognition by combining visual classification with contextual language modelling techniques. We introduced Glyph2025, a unified benchmark dataset merg-

Table 15. Corpus-BLEU scores on Franken plate test sequences.

Metric	D4 Enhanced (%)	D4 Original (%)	ResNet (%)	$\Delta_{\text{Enh-Orig}}$
BLEU-1	78.57	76.98	50.79	+1.59
BLEU-2	56.97	55.12	25.41	+1.84
BLEU-3	40.89	38.77	13.56	+2.12
BLEU-4	29.98	28.01	6.13	+1.97
BLEU (geo)	48.40	46.33	18.10	+2.07

Phase	Method	Per-pos Acc (%)	Macro F1 (%)	New Params
1 (initial)	ConvNeXt-Tiny	56.23	52.72	28.6M
1 (initial)	Glyphnet	82.66	67.68	499K
1 (final)	ResNet-18	67.08	62.83	11.2M
2†	+ Trigram (α=1.0)	89.21	79.33	0
3	+ Transformer Encoder	73.51	49.90	1.77M
4 (orig.)	+ Progressive FT	73.71	48.41	1.77M
4 (enh.)	+ LLRD FT	74.01	48.10	1.77M

Table 16. Full pipeline results across all phases. † Phase 2 evaluated on n-gram–derived sequences; others on Franken or stratified test set.

ing multiple hieroglyph corpora and incorporating substantial visual diversity across inscription styles and mediums. Through extensive experimentation, we established strong CNN baselines for isolated glyph classification and demonstrated that contextual information significantly improves recognition performance.

Our results show that statistical language modelling through n-gram re-ranking substantially improves CNN predictions, confirming that neighbouring hieroglyphs provide valuable disambiguating information. Furthermore, the Transformer Encoder architecture demonstrated that learned self-attention mechanisms can model long-range dependencies beyond the capabilities of fixed-window n-gram models. The transformer achieved notable gains in per-position accuracy by leveraging bidirectional contextual reasoning across entire sequences.

References